

Synthetic Financial Time Series Generation

GAN Benchmark: TimeGAN · QuantGAN · FinGAN

BRICS Emerging Market Indices

Collaboration	Universitat Politècnica de Catalunya (UPC)
Document type	Project Status & In-Depth Technical Review
Purpose	Paper publication preparation
Models	TimeGAN (RNN) · QuantGAN (TCN-WGAN-GP) · FinGAN (CNN-WGAN-GP)
Markets	BOVESPA · FTSE JSE · MSCI · NIFTY50 · SHANGHAI
Generated	2026-08-19

Project Status Overview

Research context · data pipeline · model status · open tasks

Synthetic Financial Time Series · UPC

Research Goal

- Benchmark three GAN architectures for synthetic financial log-return generation
- Evaluate on 5 BRICS emerging-market indices (multi-market generalisation)
- Systematically validate hyperparameters via grid search for publication
- Compare stylized-fact reproduction: fat tails, volatility clustering, long memory
- Prepare peer-reviewed paper in collaboration with UPC

Data Pipeline

- Source: 5 CSV files — BOVESPA, FTSE JSE, MSCI, NIFTY50, SHANGHAI
- Feature: daily log return $r_t = \ln(P_t/P_{t-1})$
- **No outlier clipping** applied to log returns (deliberate — see Code Audit)
- Temporal 80 / 10 / 10 split (no shuffle) — saved as Parquet files
- GAN training: windows of 128 steps pooled across all 5 markets; temporal order preserved within each window

TimeGAN

Yoon et al. (2019), NeurIPS 32. 4-phase training (autoencoder → supervisor → joint GAN) using GRU networks. BCE loss + moment matching. MinMax [0, 1] normalisation. **Current: epochs=5 (far too low — see Audit)**

QuantGAN

Wiese et al. (2020), Quantitative Finance. WGAN-GP + TCN with causal dilated convolutions. Adam $\beta_1 = 0$, $\beta_2 = 0.9$, n_critic=3, $\lambda_{gp} = 10$.

FinGAN

Custom CNN-WGAN-GP. Generator: FC → 3× ConvTranspose1d. Critic: Conv1d + LayerNorm. seq_len must be divisible by 8. Same Adam/WGAN-GP settings as QuantGAN.

Pipeline Status

- **DONE:** Reproducibility seeds; deprecated API fixes; grid search wired
- **DONE:** FinancialMetrics rewritten with time-series-aware tests
- **TODO:** Re-run pipeline to regenerate results with updated metric schema
- **TODO:** Increase TimeGAN epochs (≥ 50 –100 per phase)

The Problem — Financial Time Series from First Principles

Understanding the data before understanding the models

Synthetic Financial Time Series · UPC

What is a Financial Time Series?

A financial time series is simply a sequence of numbers measured over time. For a stock or index: $P_1, P_2, \dots, P_T$ where P_t is the closing price on day t .

Problem with raw prices: they grow over time (non-stationary), making comparisons across periods misleading. Statistical models require (approximate) stationarity.

Why Log Returns?

Instead of prices we compute the **log return** for each day:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right) = \ln P_t - \ln P_{t-1}$$

Why the logarithm?

1. **Additivity:** $r_{1 \rightarrow 3} = r_{1 \rightarrow 2} + r_{2 \rightarrow 3}$ exactly (simple returns do not add).
2. **Approximation:** $\ln(P_t/P_{t-1}) \approx (P_t - P_{t-1})/P_{t-1}$ for small moves.
3. **Stationarity:** Log returns fluctuate around zero with no long-run trend.

The Core Problem: Returns Are NOT Normal

The simplest model: $r_t \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(\mu, \sigma^2)$. This is the foundation of Black–Scholes and many classical models. **It is empirically wrong in the tails.**

Under $\mathcal{N}(0, \sigma^2)$ with $\sigma \approx 1.2\%$:

- A day with $|r| > 3\sigma \approx 3.6\%$ should occur $\approx 0.27\%$ of days $\Rightarrow$ roughly 1 day per year.
- Reality: 3–5× more frequent (**heavy tails**).
- A “5 σ crash”: under Gaussian $\approx$ once every **14 000 years**; in practice: several times per decade.

The 2008 crisis was described as a “25-sigma event” by some risk systems — probabilistically impossible under Gaussian assumptions.

What Are Stylized Facts?

Cont (2001) coined the term **stylized facts**: statistical properties that appear consistently across virtually all financial markets and time periods. They are the fingerprint of financial returns.

Any generative model claiming to produce realistic synthetic returns MUST reproduce them.

The five most important:

1. **Heavy tails:** $P(|r| > x) \sim x^{-\alpha}$ (Pareto tail, $\alpha \approx 3-5$) $\Rightarrow$ kurtosis $\gg 3$.
2. **Near-zero autocorrelation:** $\text{Corr}(r_t, r_{t+k}) \approx 0$ for $k \geq 1$ (efficient market).
3. **Volatility clustering:** $\text{Corr}(|r_t|, |r_{t+k}|) > 0$ for $k = 1, \dots, 100+$. Large moves beget large moves.
4. **Long memory:** ACF of $|r_t|$ decays as $k^{-\beta}$, $\beta \in (0, 1)$; Hurst exponent $H > 0.5$.
5. **Negative skewness:** crashes are sharper than rallies ($\text{skew}(r) < 0$ for equity indices).

Why Generate Synthetic Data?

If real data exists, why generate synthetic data?

- **Data augmentation:** deep learning models need large datasets; markets produce ~ 250 observations/year.
- **Stress testing:** banks test risk models under extreme scenarios not in historical data.
- **Privacy:** real data may be proprietary; synthetic data with the same statistical properties can be shared freely.
- **Scenario generation:** sample many plausible futures for portfolio optimisation.

The challenge: a Gaussian generator fails immediately (wrong tails, no clustering). This motivates deep generative models.

The Solution Candidate: What is a Generative Adversarial Network?

From the counterfeiter–detective analogy to the minimax formulation (Goodfellow et al., 2014)

Synthetic Financial Time Series · UPC

The Core Idea — A Two-Player Game

Goodfellow et al. (2014) proposed training **two** neural networks that compete:

Generator G : takes random noise $z \sim p_z \rightarrow$ outputs fake sample $G(z) = \tilde{x}$.

Discriminator D : given a sample (real or fake) $\rightarrow$ outputs $D(x) \in [0, 1]$ (probability of being real).

Analogy: Counterfeiter (G) makes fake banknotes; detective (D) tries to spot them. Both improve iteratively until fakes are indistinguishable.

The **minimax objective**:

$$\min_G \max_D V(D, G) = \underbrace{\mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)]}_{\text{reward } D \text{ for real}} + \underbrace{\mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]}_{\text{penalise } G \text{ when caught}}$$

Reading each term:

- D *maximises*: wants $D(x_{\text{real}}) \rightarrow 1$ and $D(G(z)) \rightarrow 0$.
- G *minimises*: wants $D(G(z)) \rightarrow 1$ (fool D).

At the **Nash equilibrium** (Goodfellow et al., 2014, Proposition 1):

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)} = \frac{1}{2} \quad \forall x \quad \implies \quad p_g = p_{\text{data}}$$

The generator has perfectly learnt the real distribution.

Critical Problem: Vanishing Gradients & Mode Collapse

Mode collapse: G learns to output only a few samples that reliably fool D , ignoring the true diversity of p_{data} .

Vanishing gradients: The GAN loss uses Jensen–Shannon (JS) divergence internally:

$$\text{JS}(p_{\text{data}} \| p_g) = \log 2 \quad \text{when } \text{supp}(p_{\text{data}}) \cap \text{supp}(p_g) = \emptyset$$

A *constant* has zero derivative with respect to G 's parameters $\Rightarrow G$ receives no learning signal.

In practice: early in training, if G produces very different-looking samples from reality, D immediately says “fake” with no useful feedback on *how* G should improve.

What Makes GANs Suitable for Financial Returns?

Unlike parametric models (GARCH, EVT), a GAN:

- Makes **no distributional assumption** about the shape of returns; learns p_{data} implicitly.
- Can model arbitrarily complex joint distributions $p(r_1, r_2, \dots, r_T)$, capturing both the marginal distribution *and* temporal dependencies.
- Scales to multivariate settings without specifying a copula.

The GAN sees sequences, not points. A window $[r_1, r_2, \dots, r_{128}]$ is the training unit. D learns to recognise temporal patterns: bursts of high volatility, calm periods. G must generate sequences exhibiting those patterns.

This is what distinguishes GAN-based generators from models that generate each timestep independently (which would produce no volatility clustering even if the marginal distribution were correct).

Three Architectural Choices in This Benchmark

Model	Backbone	Key mechanism
TimeGAN	GRU (recurrent)	Latent embedding space
QuantGAN	TCN (causal dil.)	Long receptive field via dilation
FinGAN	CNN deconvolution	Noise $\rightarrow$ sequence via upsampling

Better Training: The Wasserstein GAN and Gradient Penalty

Arjovsky et al. (2017) ICML · Gulrajani et al. (2017) NeurIPS

Synthetic Financial Time Series · UPC

A Better Distance Between Distributions

The JS divergence fails because it equals $\log 2$ (a constant) when supports do not overlap — giving G zero gradient.

Wasserstein-1 distance (“Earth Mover’s distance”):

$$W(p, q) = \inf_{\gamma \in \Pi(p, q)} \mathbb{E}_{(x, y) \sim \gamma} [\|x - y\|]$$

where $\Pi(p, q)$ is the set of all joint distributions with marginals p and q .

Physical intuition: Think of p_{data} and p_g as two piles of sand. W is the minimum total work (mass $\times$ distance) needed to reshape one pile into the other.

Why is this better?

- W is continuous and *always* provides a gradient signal, even when distributions do not overlap.
- W correlates with visual generation quality, unlike the JS loss which can plateau.

Kantorovich–Rubinstein Duality — Making W Tractable

The infimum over all joint distributions in $W(p, q)$ is computationally intractable. The key theorem (Kantorovich–Rubinstein):

$$W(p_{\text{data}}, p_g) = \sup_{\|f\|_L \leq 1} \left(\mathbb{E}_{x \sim p_{\text{data}}} [f(x)] - \mathbb{E}_{x \sim p_g} [f(x)] \right)$$

where the supremum is over all **1-Lipschitz** functions:

$$\|f\|_L \leq 1 \iff |f(x) - f(y)| \leq \|x - y\| \quad \forall x, y$$

Implication: the WGAN Critic D (now unbounded, not a probability) plays the role of f . We parameterise D with a neural network and train it to approximate the optimal f .

WGAN training objective:

$$\min_G \max_{\|D\|_L \leq 1} \mathbb{E}_{x \sim p_{\text{data}}} [D(x)] - \mathbb{E}_{z \sim p_z} [D(G(z))]$$

Gradient Penalty (Gulrajani et al., 2017)

The original WGAN enforces the Lipschitz constraint by weight clipping (crude, slow). WGAN-GP uses a differentiable penalty instead:

$$\mathcal{L}_{\text{GP}} = \lambda \cdot \mathbb{E}_{\hat{x} \sim p_{\hat{x}}} \left[\left(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1 \right)^2 \right]$$

where the interpolated point and mixing coefficient are:

$$\hat{x} = \varepsilon x_{\text{real}} + (1 - \varepsilon) x_{\text{fake}}, \quad \varepsilon \sim \mathcal{U}[0, 1], \quad \lambda = 10$$

The penalty pushes $\|\nabla D(\hat{x})\|_2 \rightarrow 1$ throughout training.

Full training losses:

$$\begin{aligned} \mathcal{L}_D &= \mathbb{E}[D(\tilde{x})] - \mathbb{E}[D(x)] + \lambda \mathcal{L}_{\text{GP}} \quad (D \text{ minimises } -\mathcal{L}_D) \\ \mathcal{L}_G &= -\mathbb{E}[D(G(z))] \end{aligned}$$

n_critic = 3: D is updated $3 \times$ per G update. This ensures D tracks the true Wasserstein distance accurately before G uses the gradient signal.

Adam Optimiser: Why $\beta_1 = 0$?

Standard Adam uses $\beta_1 = 0.9$ (momentum) and $\beta_2 = 0.999$ (RMS scaling). Gulrajani et al. (2017) recommend $\beta_1 = 0$, $\beta_2 = 0.9$.

In adversarial training, gradients *reverse direction frequently*: in one step D pushes G one way, in the next step their roles shift. Momentum ($\beta_1 > 0$) accumulates past gradients and pushes in the *wrong* direction, causing oscillations. Setting $\beta_1 = 0$ disables momentum; only the current gradient scaled by β_2 -RMS is used.

Why No BatchNorm in the WGAN-GP Critic?

BatchNorm normalises across the batch: $\hat{x}_i = (x_i - \mu_{\text{batch}}) / \sigma_{\text{batch}}$. When computing $\mathcal{L}_{\text{GP}}$, the gradient $\nabla_{\hat{x}} D(\hat{x})$ must reflect how D responds to a *single* interpolated point. BatchNorm introduces a dependency on *other samples* in the batch, corrupting the gradient norm calculation and breaking the Lipschitz

enforcement.

Solution: use **LayerNorm** (normalises per sample across features), which has no batch dependency. Both QuantGAN and FinGAN use LayerNorm in the critic.

TimeGAN: Making the GAN Time-Aware

Yoon, Jarrett & van der Schaar (2019) NeurIPS 32 — GRU + embedding space + 4-phase training

Synthetic Financial Time Series · UPC

Why a Plain GAN Fails on Time Series

A basic GAN generates each window independently from a noise vector. It *can* match the marginal distribution $p(r_t)$ (correct tails, correct scale), but has no mechanism to enforce that consecutive steps within a window exhibit temporal patterns like volatility clustering.

Analogy: Writing a novel by choosing each word independently from a realistic vocabulary. Individual words are real English, but the sequence is incoherent.

What is a GRU? (From Scratch)

A **GRU** (Gated Recurrent Unit) processes a sequence step by step, maintaining a “hidden state” h_t — a running summary of all past inputs.

At each step t , given x_t (current input) and h_{t-1} (previous summary):

$$\begin{aligned} r_t &= \sigma(W_r[h_{t-1}, x_t]) && \text{reset gate} \\ z_t &= \sigma(W_z[h_{t-1}, x_t]) && \text{update gate} \\ \tilde{h}_t &= \tanh(W[h_t \odot h_{t-1}, x_t]) && \text{candidate state} \\ h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t && \text{new hidden state} \end{aligned}$$

The update gate z_t controls how much past information to keep vs. replace. The key property: h_t is a *compressed summary* of all inputs so far.

Key Innovation: Generating in Embedding Space

TimeGAN trains the GAN in a **learned latent space** $\mathcal{H}$, not in the raw return space $\mathcal{X}$.

Why? Raw returns are noisy. A smooth latent space is easier to model with a GAN. Moreover, a *Supervisor* network is trained to predict h_{t+1} from h_t , embedding the temporal dynamics into $\mathcal{H}$. G then generates sequences in $\mathcal{H}$ that follow these learned dynamics.

The Four-Phase Training Algorithm

Phase 1 — Autoencoder (epochs // 2 steps):

- Embedder $e : \mathcal{X} \rightarrow \mathcal{H}$; Recovery $\hat{r} : \mathcal{H} \rightarrow \tilde{\mathcal{X}}$
- $\mathcal{L}_R = \|X - \hat{r}(e(X))\|^2$ (reconstruction)

Phase 2 — Supervisor (epochs // 2 steps):

- Supervisor $s : \mathcal{H} \rightarrow \hat{\mathcal{H}}$ (step-ahead prediction in latent space)
- $\mathcal{L}_S = \|H_{t+1} - s(H_t)\|^2$

Phase 3 — Joint Adversarial Training (epochs steps):

$$z \rightarrow G(z) = \hat{H} \rightarrow s(\hat{H}) \rightarrow \hat{r}(\cdot) \rightarrow \tilde{X}$$

$$\mathcal{L}_G = \mathcal{L}_U + 100\sqrt{\mathcal{L}_S} + 100\mathcal{L}_V$$

$$\mathcal{L}_U = -\mathbb{E}[\log D(s(G(z)))] \quad (\text{adversarial})$$

$$\mathcal{L}_V = |\mu_H - \mu_{\hat{H}}| + |\sigma_H - \sigma_{\hat{H}}| \quad (\text{moments})$$

$$\text{Discriminator: } \mathcal{L}_D = \text{BCE}(D(H_{\text{real}}), 1) + \text{BCE}(D(H_{\text{fake}}), 0)$$

Critical Issue: Training Budget

With the current setting **epochs = 5**:

- Phase 1 gets $5//2 = 2$ gradient steps.
- Phase 2 gets $5//2 = 2$ gradient steps.

A GRU cannot converge in 2 steps. The embedding space $\mathcal{H}$ is essentially random noise $\Rightarrow$ Phase 3 trains a GAN on top of garbage.

Yoon et al. (2019) use **5 000–10 000 iterations per phase**. TimeGAN’s poor ranking (avg_rank = 2.36) almost certainly reflects this budget, not its architecture.

Fix required: epochs ≥ 50 (or add explicit per-phase iteration counts).

QuantGAN & FinGAN: Convolutional Approaches to Time Series Generation

Building from convolutions to causal dilated TCNs to CNN-WGAN-GP generators

Synthetic Financial Time Series · UPC

What is a Convolution? (From Scratch)

A 1D convolution slides a small filter (kernel) of size K over a sequence:

$$\text{out}[t] = \sum_{k=0}^{K-1} w_k \cdot \text{in}[t - k]$$

Think of it as a magnifying glass that scans the time series, highlighting patterns of a specific shape. The weights $w_0, \dots, w_{K-1}$ are learnt from data.

Causal constraint (no future leakage): only inputs at $t, t-1, \dots, t-(K-1)$ are used.

Dilated convolution (skip steps to see further back cheaply):

$$\text{out}[t] = \sum_{k=0}^{K-1} w_k \cdot \text{in}[t - k \cdot d]$$

With dilation $d = 1, 2, 4, 8, \dots$ and $K = 2$: receptive field grows as $1 + \sum_l 2^l$, doubling per layer with no extra parameters.

QuantGAN — TCN-based WGAN-GP (Wiese et al., 2020)

Key innovation: replace GRU with a Temporal Convolutional Network (TCN) — fully parallelisable, no vanishing-gradient-across-time problem.

Generator: z (noise_dim=100) $\rightarrow$ Linear $\rightarrow$ reshape $(C, T/4) \rightarrow$ ConvTranspose1d $\times 2$ (stride 2, $2\times$ upsample) $\rightarrow$ TCN (3 residual blocks, dilations 1/2/4) $\rightarrow$ Conv1d(1) $\rightarrow$ Tanh $\rightarrow \tilde{x}$

Critic: x ($T \times 1$) $\rightarrow$ permute $\rightarrow$ TCN (3 blocks) $\rightarrow$ AdaptiveAvgPool1d(1) $\rightarrow$ Linear(128) $\rightarrow$ LeakyReLU $\rightarrow$ Linear(1)

AdaptiveAvgPool averages the entire temporal dimension, making the critic size-agnostic and capturing global sequence statistics.

FinGAN — CNN Deconvolution WGAN-GP

Transposed convolution: the “reverse” of a strided convolution; inserts zeros between inputs then applies a kernel. Stride 2 doubles the output length. Three such layers: $T/8 \rightarrow T/4 \rightarrow T/2 \rightarrow T$ (hence seq_len divisible by 8).

Generator: $z \rightarrow$ Linear+BN $\rightarrow$ reshape $(C, T/8) \rightarrow$ ConvTranspose1d $\times 3$ (each stride 2, BN, LeakyReLU) $\rightarrow$ Tanh

Critic: Conv1d $\times 3$ with LayerNorm (not BatchNorm; see page 5) + LeakyReLU $\rightarrow$ Flatten $\rightarrow$ Linear(128) $\rightarrow$ Linear(1)

Architectural Comparison

	TimeGAN	QuantGAN	FinGAN
Backbone	GRU (recurrent)	TCN (parallel)	CNN deconv
Temporal	Step-by-step hidden state	Dilated receptive field	Global upsample
Normalisation	MinMax [0, 1]	MinMax [-1, 1]	MinMax [-1, 1]
Loss	BCE + moments	WGAN-GP	WGAN-GP
avg_rank	2.36	1.79	1.85

TimeGAN’s poor ranking is almost certainly due to epochs=5, not its architecture.

What We Measure: The 10 Stylized Facts of Financial Returns

Cont (2001) · Mandelbrot (1963) · Ding, Granger & Engle (1993)

Synthetic Financial Time Series · UPC

#	Stylized Fact	In Plain English	Mathematical Statement	Our Metric
1	Heavy tails	Extreme events occur far more often than a bell curve predicts.	$P(r > x) \sim x^{-\alpha}$, $\alpha \approx 3-5$ (Pareto); $\text{kurtosis}(r) \gg 3$	Kurtosis diff · tail-index diff
2	Near-zero autocorrelation	Knowing today's direction gives no useful information about tomorrow's.	$\text{Corr}(r_t, r_{t+k}) \approx 0$ for $k \geq 1$ (weak-form efficiency)	ACF returns MAE
3	Volatility clustering	Large moves tend to be followed by large moves (Mandelbrot, 1963).	$\text{Corr}(r_t , r_{t+k}) > 0$ for $k = 1, \dots, 100+$	ACF returns MAE · ARCH-LM stat diff
4	Long memory in volatility	The clustering effect persists for hundreds of trading days.	$\text{ACF}(r_t) \sim k^{-\beta}$, $\beta \in (0, 1)$; Hurst exponent $H > 0.5$	Hurst exponent diff
5	Gain/loss asymmetry	Crashes are sharper and more extreme than equivalent-size rallies.	$\text{skewness}(r) < 0$ for equity indices; left tail heavier than right	Skewness diff
6	ARCH effects	Return variance changes over time; it is not constant.	$\text{Var}(r_t \mid \mathcal{F}_{t-1}) = \sigma_t^2$ (time-varying, not constant σ^2)	ARCH-LM test (Engle, 1982)
7	Conditional heavy tails	After removing time-varying variance, residuals are still non-Gaussian.	$\varepsilon_t = r_t / \sigma_t$ still has kurtosis $\gg 3$ (GARCH standardised residuals)	Kurtosis diff on residuals (ext.)
8	Extreme events	Very large days occur far more frequently than Gaussian probability predicts.	% of days with $ r > 2\sigma$ exceeds the 5% expected under $\mathcal{N}(0, 1)$	Extreme events diff
9	Distributional match	The full shape of the return distribution — not just its tails — must be reproduced.	$W_1(p_g, p_{\text{data}}) \approx 0$; $\hat{F}_{\text{syn}} \approx \hat{F}_{\text{real}}$ (quantile-by-quantile)	Wasserstein · Quantile MSE · Energy distance
10	Indistinguishability	A classifier trained to separate real from synthetic should perform at chance level.	$\text{AUC}(\text{classifier}) \rightarrow 0.5$; $p_g = p_{\text{data}}$ at GAN optimum	Discriminative AUC

Evaluation Metrics Part 1 — Why Standard Statistical Tests Fail

The i.i.d. assumption: what it means, why financial series violate it, mathematical consequences

Synthetic Financial Time Series · UPC

What Does i.i.d. Mean?

A sample $\{X_1, X_2, \dots, X_n\}$ is **i.i.d.** (independent and identically distributed) if:

- *Identically distributed*: every X_i has the same distribution F .
- *Independent*: knowing $X_1, \dots, X_{t-1}$ gives **no** information about X_t .

Coin flips are i.i.d. Financial returns are **not**:

- $|r_{t+1}|$ is correlated with $|r_t|$ (volatility clustering).
- $\text{Var}(r_t | \mathcal{F}_{t-1}) = \sigma_t^2 \neq \text{const}$ (ARCH effects).
- $\text{ACF}(|r_t|)$ is still positive at lag $k = 100+$ (long memory).

Most classical statistical tests are *derived* under the i.i.d. assumption.

The Kolmogorov–Smirnov Test and Its Failure

The two-sample KS statistic:

$$D_{n,m} = \sup_x |\hat{F}_n(x) - \hat{G}_m(x)|$$

Under H_0 (same distribution) **and** i.i.d.:

$$\sqrt{\frac{nm}{n+m}} \cdot D_{n,m} \xrightarrow{d} K \quad (\text{Kolmogorov distribution})$$

For *dependent* series, the CLT underlying this limit changes:

$$\text{Var}\left(\frac{1}{n} \sum_{t=1}^n X_t\right) = \frac{\sigma^2}{n} \left(1 + 2 \sum_{k=1}^{\infty} \rho_k\right)$$

This defines the **effective sample size**:

$$n_{\text{eff}} = \frac{n}{1 + 2 \sum_{k=1}^{\infty} \rho_k}$$

For BRICS |returns|: $\sum_{k=1}^{100} \rho_{|r|}(k) \approx 5\text{--}15$, so:

$$n_{\text{eff}} \approx \frac{n}{11} \quad \text{to} \quad \frac{n}{31}$$

Consequence: The KS test uses n but the effective information is only n_{eff} . $\Rightarrow$ Test statistic is *inflated*. $\Rightarrow$ p -values are *too small* (anti-conservative). $\Rightarrow$ We over-reject H_0 even when the marginal distributions match, because temporal dependence is mistaken for distributional difference.

What Can the KS Test Still Tell Us?

Despite its limitations, the KS test is not useless:

- It tests the *marginal* distribution $F(x) = P(r \leq x)$, which is well-defined even for stationary dependent series.
- A GAN producing the wrong scale or non-financial-looking returns will fail the KS test.

Strategy used in this project:

1. Report the KS *statistic* (not the p -value) as a marginal-distribution distance measure.
2. Use **Wasserstein distance** $W_1(p_g, p_{\text{data}})$ as the primary distributional metric — a proper metric that is more stable under temporal dependence.
3. Add time-series-aware tests (ARCH-LM, Hurst exponent, discriminative AUC) to capture what KS cannot.

Why Pointwise MSE/MAE Are Wrong

The evaluation previously used:

$$\text{MSE} = \frac{1}{T} \sum_{t=1}^T (r_{\text{real}}(t) - r_{\text{syn}}(t))^2$$

This compares the t -th real return to the t -th synthetic return. But real and synthetic series are **independently generated** — there is no meaningful alignment between their temporal indices.

Analogy: Measuring the “error” between two random walks by comparing step 1 to step 1, step 2 to step 2, etc. This measures nothing about distributional similarity.

Fix: Use **Quantile MSE** (compares sorted distributions) and **Energy distance** (a proper metric on the space of distributions). See next page.

Welch’s t -test: A Narrower Failure

Welch’s test checks $H_0 : \mu_{\text{real}} = \mu_{\text{syn}}$:

$$t = \frac{\bar{X} - \bar{Y}}{\sqrt{s_X^2/n + s_Y^2/m}}$$

For log returns, $\rho_k(r_t) \approx 0$ for $k \geq 1$, so Welch’s test is *approximately valid* for testing mean equality. However, it tests **only the mean** — providing no information about

tails, volatility clustering, or any other stylized fact. Using it as the primary evaluation metric tells us almost nothing about financial realism.

Evaluation Metrics Part 2 — New Time-Series-Aware Tests Added

ARCH-LM · Hurst exponent · Energy distance · Discriminative AUC · Quantile MSE

Synthetic Financial Time Series · UPC

1. ARCH-LM Test — Volatility Clustering

What it tests: H_0 = no ARCH effects (homoskedasticity).

Procedure (Engle, 1982):

1. Regress squared residuals on their own lags:

$$\hat{\varepsilon}_t^2 = \alpha_0 + \sum_{j=1}^q \alpha_j \hat{\varepsilon}_{t-j}^2 + v_t$$

2. Compute $LM = n \cdot R^2$, where R^2 is from the auxiliary regression.
3. Under H_0 : $LM \sim \chi^2(q)$.
4. If $p < 0.05$: reject $H_0 \Rightarrow$ ARCH effects present.

How we use it: BRICS real returns consistently show ARCH ($p \ll 0.05$). A good GAN's synthetic returns should also show ARCH. Our metric: $|LM_{\text{real}} - LM_{\text{syn}}| \rightarrow 0$.

2. Hurst Exponent — Long Memory

What it captures: whether |returns| have the slow ACF decay (long memory, Stylized Fact 4) that real financial series exhibit.

The Hurst exponent H characterises the scaling of the rescaled range R/S :

$$\mathbb{E} \left[\frac{R_n}{S_n} \right] \sim c \cdot n^H$$

where $R_n = \max_t(\text{partial sums}) - \min_t(\text{partial sums})$ and $S_n = \text{std}(\{r_t\})$.

Interpretation:

- $H = 0.5$: Brownian motion (no memory, i.i.d. increments).
- $H > 0.5$: persistent long memory (trends reinforce).
- $H < 0.5$: mean-reverting (anti-persistent).

For log returns r_t : $H \approx 0.5$ (efficient). For $|r_t|$: $H \approx 0.6$ – 0.7 .

Our metric: $|H_{\text{real}} - H_{\text{syn}}| \rightarrow 0$.

3. Energy Distance — A Proper Distributional Metric

The energy distance between distributions P and Q (Székely & Rizzo, 2004):

$$E(P, Q) = 2 \mathbb{E} \|X - Y\| - \mathbb{E} \|X - X'\| - \mathbb{E} \|Y - Y'\|$$

where $X, X' \sim P$ and $Y, Y' \sim Q$ are independent copies.

Key properties:

- $E(P, Q) \geq 0$, with $E(P, Q) = 0$ iff $P = Q$. A *true metric* on distributions.
- Does not require a density function or independence.
- Uses only marginal expectations (no joint γ); avoids the i.i.d. problem of KS.

Estimated using subsampled pairs ($n = \min(500, N)$), $O(n^2)$ cost.

4. Discriminative AUC — The Holistic Score

Procedure (TSGBench, Ang et al., 2023):

1. Extract rolling-window features from both real and synthetic (mean, std, skewness, kurtosis, $|\bar{r}|$, $\max |r|$).
2. Label real windows 0, synthetic windows 1.
3. Train logistic regression; report AUC via 5-fold CV.

AUC = 0.5: distributions indistinguishable. AUC = 1.0: trivially separable.

This is the best single summary because it captures both distributional *and* temporal structure simultaneously. A GAN with the correct marginal but wrong temporal structure will still show AUC > 0.5.

5. Quantile MSE — Proper Distribution Shape Comparison

Pointwise MSE is meaningless (no index alignment). Quantile MSE compares empirical quantile functions:

$$\text{QMSE} = \frac{1}{K} \sum_{k=1}^K (Q_{\text{real}}(\alpha_k) - Q_{\text{syn}}(\alpha_k))^2, \quad \alpha_k = \frac{k}{K+1}, \quad K = 99$$

Interpretation: sort both series independently; compare the k -th smallest real return to the k -th smallest synthetic return. This is the only meaningful way to compare two unaligned distributions.

$\text{QMSE} = 0 \Leftrightarrow \hat{F}_{\text{real}} = \hat{F}_{\text{syn}}$. Equivalent to the L^2 distance between empirical CDFs. Captures tail differences (1st, 5th percentile) critical for finance.

Code Audit: Issues Found, Mathematical Justification & Changes Applied

3_4_integrated_pipeline.ipynb — reviewed and updated July 2026

Synthetic Financial Time Series · UPC

[FIXED] KS / Welch tests assumed i.i.d. — CRITICAL

$n_{\text{eff}} = n / (1 + 2 \sum_k \rho_k) \ll n$ for financial series (the sum of ACF of $|\text{returns}|$ is $\approx 5\text{--}15$ for BRICS). The KS statistic is inflated, making p -values anti-conservative. **Fix:** added ARCH-LM test, Hurst exponent, energy distance, and discriminative AUC. KS statistic retained with explicit caveat; Wasserstein distance used as primary distributional metric.

[FIXED] Pointwise MSE/MAE had no semantic meaning

MSE compared $r_{\text{real}}(t)$ to $r_{\text{syn}}(t)$, but real and synthetic are independently generated: no temporal alignment exists between indices. **Fix:** replaced with Quantile MSE (L^2 distance between empirical quantile functions) and Energy distance (Székely & Rizzo, 2004 — a proper metric on the space of distributions).

[DECISION] No outlier clipping applied to log returns — deliberate methodological choice

The preprocessing notebook computed 0.5th/99.9th percentile bounds on raw price columns, but the clip line was commented out and was *never* applied to log returns. This is kept intentionally. Clipping at 0.5th/99.5th removes exactly the observations that determine kurtosis, the tail index, and the “% of $|r| > 2\sigma$ ” metric — all three appear in the evaluation table. Truncating the tail and then reporting that models reproduce tails is circular. Adams et al. (2019, *Financial Management*) show that winsorizing/trimming can worsen rather than fix distributional misfit. LeBaron’s EVT work finds BRICS emerging markets have systematically fatter tails than developed markets — the very property motivating the BRICS market choice.

[FIXED] Ljung-Box test used deprecated statsmodels API

`acorr_ljungbox(..., return_df=False)` returns a DataFrame in statsmodels ≥ 0.13 , not a tuple. Accessing `lb[1][-1]` raised `TypeError`. **Fix:** `return_df=True, lags=[n]` (list syntax), accessed via `.iloc[-1][“lb_pvalue”]`.

[FIXED] ResidualBlock class defined but never used in FinGAN cell

`class ResidualBlock(nn.Module)` was defined at the top of the FinGAN cell but neither `FinGAN_Generator` nor `FinGAN_Critic` reference it. Dead code misleads readers into thinking residual connections are active. **Fix:** class removed entirely.

[FIXED] Grid search `score_cols` included pointwise MSE and MAE

The composite ranking used to select hyperparameter configurations included `mse` and `mae` — meaningless as explained above. **Fix:** removed from `score_cols`; added `hurst_diff`, `energy_distance`, `arch_stat_diff`.

[OK] WGAN-GP gradient penalty computation is correct (QuantGAN & FinGAN)

$\hat{x} = \varepsilon x_{\text{real}} + (1 - \varepsilon)x_{\text{fake}}$; `requires_grad_(True)`; gradients computed via `torch.autograd.grad` with `create_graph=True`. `fake.detach()` prevents spurious gradient accumulation. Correctly implements Gulrajani et al. (2017), Equation 3.

[OK] Adam $\beta_1 = 0$, $\beta_2 = 0.9$ for WGAN-GP — correct

$\beta_1 = 0$ disables first-moment (momentum) accumulation. In adversarial training, gradients reverse direction frequently; momentum from the previous step destabilises training. $\beta_2 = 0.9$ retains RMS adaptive scaling. Recommended explicitly by Gulrajani et al. (2017).

[CONCERN] TimeGAN training budget of `epochs=5` is scientifically insufficient

Phases 1 and 2 each receive $5//2 = 2$ gradient steps. A GRU cannot converge in 2 steps; the embedding space $\mathcal{H}$ is effectively random. Yoon et al. (2019) use 5 000–10 000 iterations per phase. TimeGAN’s poor ranking (`avg_rank = 2.36`) almost certainly reflects this budget, not its architecture. Requires fix: `epochs` ≥ 50 (or add explicit per-phase iteration counts).

Hyperparameter Grid Search: Why, How, and What It Documents

Required for publication: systematic evidence that parameters were tested before selection

Synthetic Financial Time Series · UPC

Why Grid Search is Required for Publication

A common reviewer critique: “how were hyperparameters chosen?” If the answer is “we tried a few and picked the best-looking result”, reviewers rightly question reproducibility.

Grid search provides a **paper trail**:

- Search space is defined *ex ante* (before seeing results).
- Selection criterion is objective (composite rank of stylized-fact metrics).
- Every candidate configuration is logged to a CSV (appendix).
- The winning configuration is retrained at full epochs.

This turns hyperparameter choice from a hidden decision into a documented, reproducible scientific step.

The Search Protocol

For each (model, config) combination:

1. Set `epochs=3` (reduced budget for speed), `seed=42`.
2. Train on pooled BRICS training windows.
3. Generate synthetic series; score with `FinancialMetrics`.
4. Rank each candidate on each metric (rank 1 = best).
5. $\text{composite_score}_i = \frac{1}{|\text{score_cols}|} \sum_m \text{rank}_m(i)$
6. Best config → retrain at full epochs for final results.

Composite Scoring Formula

`score_cols` = {wasserstein, ks_statistic, kurtosis_diff, skewness_diff, acf_returns_mae, acf_absolute_mae, tail_index_diff, hurst_diff, energy_distance, arch_stat_diff}

Excluded from scoring:

- `mse`, `mae` — pointwise metrics (no temporal alignment, meaningless)
- `ks_pvalue` — correlated with `ks_statistic`; double-counting

Search Space (8 candidates = 2^3 per model)

Model	Parameter	Values
TimeGAN	hidden_dim	{24, 48}
	num_layers	{2, 3}
	lr	$\{10^{-3}, 5 \times 10^{-4}\}$
QuantGAN	lr	$\{10^{-4}, 2 \times 10^{-4}\}$
	n_critic	{3, 5}
	lambda_gp	{5, 10}
FinGAN	lr	$\{10^{-4}, 2 \times 10^{-4}\}$
	n_critic	{3, 5}
	base_channels	{32, 64}

Reproducibility

Each candidate calls `torch.manual_seed(seed)` and `np.random.seed(seed)`. Same config + same seed ⇒ identical weight initialisation and training trajectory. Results saved to: `grid_search_results/{Model}_grid_search.csv`. This CSV is the paper appendix for hyperparameter documentation.

Recommended Extensions for Publication

- Extend to 20–30 configs × 10 epochs per candidate (currently 8×3).
- Use Bayesian optimisation (Optuna) for more efficient exploration.
- Ablation table: vary one parameter at a time, fix all others; report marginal effect on composite score.
- 5-fold cross-validation across market splits for robust scoring.
- Report best config with 95 % CI on composite score across seeds.

Ranked Next Steps for Paper Publication

Priority order based on impact on scientific rigour and reviewability

Synthetic Financial Time Series · UPC

[CRITICAL] 1 — Re-run the full pipeline and regenerate all results

Evaluation metrics schema changed (ARCH-LM, Hurst, discriminative AUC, energy distance added; pointwise MSE/MAE removed from ranking). All files in `thesis_results/` are stale. Must regenerate before any comparative analysis or figure creation for the paper.

[CRITICAL] 2 — Fix TimeGAN training budget (epochs=5 is scientifically unsound)

Phases 1 and 2 each get 2 gradient steps. Yoon et al. (2019) use 5 000–10 000 iterations per phase. Recommendation: set `epochs` ≥ 50 or add explicit per-phase iteration count parameters.

[HIGH] 3 — Robustness test on a different asset class (FX or Crypto)

Supervisor request: train the best GAN (QuantGAN) on BRICS, then evaluate on EUR/USD or BTC/USD daily log returns. Validates out-of-distribution generalisation — a key contribution distinguishing the paper from single-asset GAN studies.

[HIGH] 4 — Add or discuss diffusion model baseline

Takahashi & Mizuno (2025, *Quantitative Finance* 25(10)) showed diffusion models outperform GANs on several stylized-fact metrics. Reviewers will ask why diffusion was not included. At minimum: a dedicated Discussion section. Best: include a simple DDPM or score-matching baseline.

[HIGH] 5 — Document all LLM parameters for reproducibility

DeepSeek experiments: report temperature, `top_p`, `max_tokens`, system/user prompt verbatim. LoRA fine-tuning: $r = 8$, $\alpha = 16$, target modules (`q_proj`, `v_proj`), dropout, epochs, batch size, gradient accumulation steps.

[MEDIUM] 6 — Expand grid search and publish full ablation table

Current: 8 configs $\times$ 3 epochs. For publication: extend to 20–30 configs $\times$ 10 epochs; add ablation table showing marginal effect of each hyperparameter.

[MEDIUM] 7 — Update literature review with 2024/2025 papers

Must cite: Meldrum et al. (2025) arXiv:2510.26076; SFAG (2026) arXiv:2601.12990; TSGBench Ang et al. (2023) PVLDB 17(3); Chronos Ansari et al. (2024) arXiv:2403.07815; LLMTime Gruver et al. NeurIPS 2023; Forging TS Hamdouche et al. (2025) arXiv:2505.17103.

[LOW] 8 — LLM diversity analysis: correlation between independent generation runs

Generate 10+ independent synthetic series from each LLM. Compute pairwise Wasserstein distances and correlations. Mode collapse → high correlation. Diversity → low. Addresses the supervisor's question about LLM output diversity.

[LOW] 9 — Fix cosmetic filename strip in `validate_models()`

`validate_models()` calls `.replace("_val.parquet","")` but files are named `"_valid.parquet"`. Display names show as `"BOVESPA_valid"` instead of `"BOVESPA"`. One-line fix; no effect on results.